



Tecnológico de Monterrey

Instituto Tecnológico y de Estudios Superiores de Monterrey
Campus Puebla

TE3002B: Implementación de robótica inteligente (Gpo 501)

Profesor: Dr. Rigoberto Cerino Jiménez

Reto semanal 6. Manchester Robotics

Equipo *I miss NWJNS*

Tania Sofía Arrazola Bello	A01738124
Iñaki Ahedo Madrid	A01738231
Marcos Allen Martínez Cortés	A01737939

25 de mayo de 2026

Resumen

En el presente *Challenge 7*, se desarrolló e implementó un sistema de visión computacional basado en aprendizaje profundo para el reconocimiento automatizado de señales de tránsito, diseñado para operar de manera distribuida con el Puzzlebot bajo el entorno de ROS 2. El sistema es capaz de detectar, clasificar y delimitar espacialmente en tiempo real siete directivas viales: `ahead_only`, `give_way`, `roadwork_ahead`, `roundabout`, `stop`, `turn_left_ahead` y `turn_right_ahead`.

La metodología de preparación de datos abarcó la adquisición sistemática de imágenes y su posterior etiquetado empleando la plataforma *Roboflow*. Para el preprocesamiento, los fotogramas fueron redimensionados a la resolución nativa de la cámara (640×480) y sometidos a técnicas de aumento de datos (*Data Augmentation*) para incrementar la invarianza del modelo ante cambios de iluminación o perspectiva en el entorno físico. El modelo predictivo se entrenó utilizando la arquitectura *convolucional* de YOLOv8.

El sistema consistió de una red neuronal donde el despliegue de esta se estructuró un nodo en ROS 2 que emplea la librería *CvBridge* para suscribirse al flujo de video crudo del robot `/video_source/raw`. Mediante un temporizador, el nodo ejecuta la inferencia a una frecuencia estable de 20 Hz, publicando los resultados de forma estructurada en el tópico `/yolo/detecciones` a través de mensajes estándar de tipo `String`, conteniendo la clase y sus coordenadas delimitadoras *Bounding Boxes*. De manera simultánea, procesa y publica las imágenes anotadas en el tópico `/yolo/inference_result` para facilitar la telemetría y retroalimentación visual remota. Finalmente, la integración física y la descarga computacional se resolvieron estableciendo una red de comunicación inalámbrica mediante la configuración de la variable `ROS_DOMAIN_ID`, permitiendo separar la carga de inferencia del hardware del robot.

Objetivos

El objetivo principal de este proyecto es desarrollar un sistema de detección de objetos basado en redes neuronales convolucionales para el Puzzlebot utilizando YOLO y ROS2. El sistema integra un modelo entrenado con un conjunto de datos propios para identificar siete señales de tránsito (a escala) en tiempo real, funcionando como base para la toma de decisiones autónoma durante el seguimiento de una trayectoria. Para alcanzar ese propósito, se establecieron los siguientes objetivos:

- **Recopilación y preprocesamiento de datos:** Construir y etiquetar un conjunto de datos personalizado para las siete señales de tránsito teniendo más de mil imágenes con distintas variaciones visuales para optimizar el aprendizaje de la red neuronal frente a diferentes entornos.
- **Entrenamiento del modelo:** Entrenar un modelo de detección de objetos mediante el uso del algoritmo de YOLOv8, ajustando los hiper parámetros para

lograr un equilibrio entre la precisión predictiva, extracción de características y velocidad de inferencia.

- **Nodo de percepción:** Implementar un nodo con la arquitectura de ROS2 que capture el video del robot y se ejecute el modelo previamente entrenado para constantemente realizar inferencias y detectar el objeto junto con sus coordenadas.
- **Gestión de Datos mediante ROS 2.** Administrar de manera eficiente el flujo de información y estados del sistema a través de los tópicos asegurando una comunicación síncrona y robusta entre los nodos de percepción y el módulo de control.
- **Optimización y Robustez del Sistema.** Garantizar la fiabilidad de las detecciones frente a perturbaciones visuales, asegurando que el modelo tenga inferencias correctas incluso ante distintas iluminaciones o desenfoques por movimiento, además que tenga un nivel de tolerancia ante predicciones falsas o con poca probabilidad.

Introducción

El presente reto aborda el reentrenamiento de una red neuronal, particularmente de la arquitectura YOLO, con el objetivo de que se puedan identificar las señales de tránsito en el entorno a través del sistema de visión artificial de un robot móvil, utilizando el framework ROS 2 e implementando un modelo de *deep learning* con robustez para la detección en tiempo real a través de una cámara embebida en el robot.

Aprendizaje profundo y redes neuronales

El aprendizaje profundo (*deep learning*) es una rama del *machine learning* impulsada por redes neuronales artificiales multicapa (o profundas), cuyo diseño se inspira en la estructura del cerebro humano [1, 2]. Se define específicamente por el entrenamiento de modelos que cuentan con al menos 4 capas interconectadas de “neuronas” o nodos; aunque las arquitecturas modernas son mucho más profundas, llegando a tener cientos o miles de capas ocultas entre la entrada y la salida [1, 2].

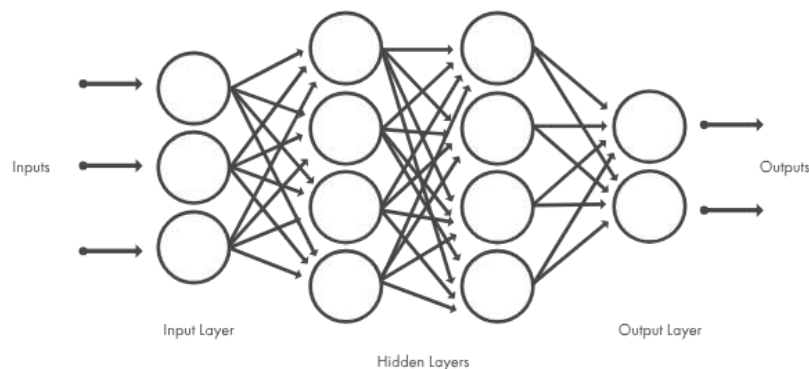


Figura 1: Arquitectura de una red neuronal típica [2].

El aprendizaje profundo funciona mediante redes neuronales artificiales estructuradas en capas: una de entrada, múltiples capas ocultas (donde ocurre el aprendizaje) y una de salida que calcula la predicción. Cada nodo realiza una operación matemática no lineal llamada función de activación para procesar datos y detectar patrones complejos. Durante el entrenamiento, las conexiones entre nodos se ajustan mediante pesos y sesgos para reducir el error del modelo. Esto se logra de forma iterativa utilizando conjuntamente dos algoritmos: la retropropagación (que calcula el gradiente del error hacia atrás) y el descenso de gradiente (que determina cómo ajustar los parámetros) [1, 2].

Las innovaciones arquitectónicas han dado lugar a los siguientes tipos de modelos: redes neuronales convolucionales (CNN), redes neuronales recurrentes (RNN) y su variante LSTM, modelos transformadores, modelos Mamba, decodificadores, modelos de difusión, redes generativas adversativas (GAN), redes neuronales de grafos (GNN), entre otros [1, 2]. Estas arquitecturas se implementan en diversos campos tecnológicos e industriales tales como visión artificial y procesamiento de imágenes, conducción autónoma, procesamiento del lenguaje natural (PLN), IA generativa, investigación médica, procesamiento de señales, robótica y control de sistemas [1, 2].

Redes neuronales convolucionales (CNN)

Las redes neuronales convolucionales (CNN o *ConvNets*) son arquitecturas diseñadas para el procesamiento de datos con estructura espacial (multidimensional), que destacan en tareas de visión artificial debido a su capacidad para aprender de manera automática y jerárquica, identificando desde patrones simples hasta objetos complejos [3].

El funcionamiento de una CNN se basa en una secuencia estructurada de capas que transforman los datos de entrada [4]. El proceso inicia en la capa convolucional, donde un filtro o *kernel* se desplaza sobre la imagen generando mapas de características que resaltan patrones específicos según hiperparámetros como el *stride* (desplazamiento) y el *padding* (relleno de bordes) [3]. Posteriormente, se aplica una capa de activación (usualmente ReLU) para introducir no linealidad, seguida de una capa de agrupación o *pooling* (como *Max Pooling*), que reduce las dimensiones espaciales para acelerar el cómputo y mitigar el sobreajuste [3, 4].

Finalmente, la información procesada se unifica en una capa de aplanado (flattening), que convierte los mapas multidimensionales en un vector unidimensional [4]. Este vector ingresa a la capa totalmente conectada (*fully connected*), donde todos los nodos se vinculan para realizar el razonamiento de alto nivel [4]. El recorrido concluye en la capa de salida, que utiliza funciones como *Softmax* para asignar las probabilidades de clasificación finales [3, 4].

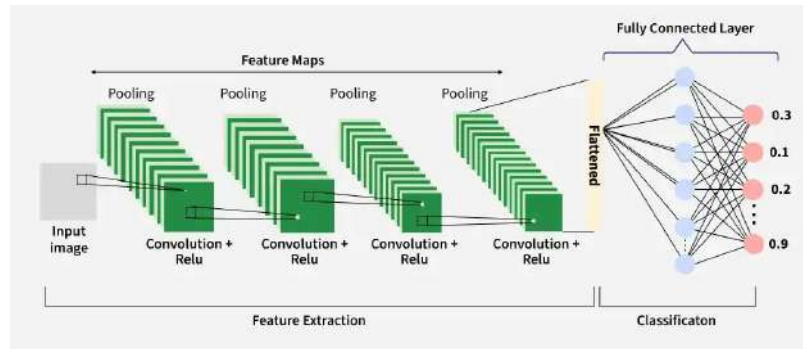


Figura 2: Arquitectura de redes neuronales convolucionales [4].

Data augmentation

El aumento de datos (*data augmentation*) es una técnica de preprocesamiento en *machine learning* que genera copias modificadas de datos existentes para incrementar la diversidad y el tamaño de un conjunto de entrenamiento sin recolectar nuevas muestras reales [5, 6]. Su objetivo principal es reducir el sobreajuste, corregir clases desbalanceadas y mejorar la generalización y robustez de los modelos [5, 6]. A diferencia de los datos sintéticos (completamente artificiales), el aumento de datos solo aplica transformaciones a datos pre existentes manteniendo sus etiquetas originales [5].

Las estrategias varían según el tipo de datos; en imágenes se suelen emplear cambios geométricos (rotación, giros, escalado) y fotométricos (ajustes de brillo, contraste o ruido gaussiano), además de filtros de *kernel* y borrado aleatorio [5, 6]. Actualmente, librerías como TensorFlow, PyTorch, Keras y Albumentations permiten implementar técnicas de aumento de datos, e incluso automatizarlas, para optimizar tareas complejas de clasificación y segmentación [5, 6].

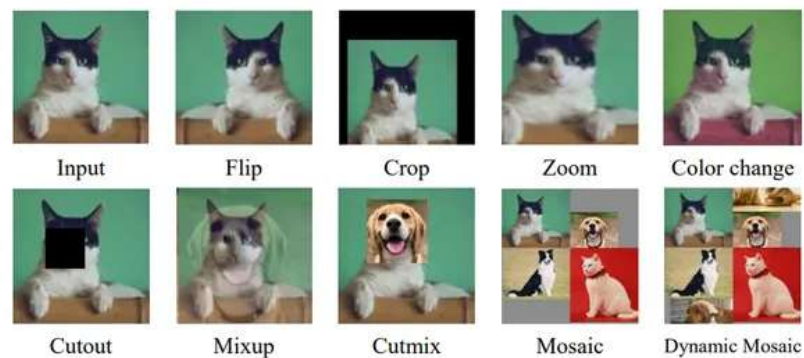


Figura 3: Ejemplo de Data Augmentation <https://www.ibm.com/es-es/think/topics/data-augmentation>

Aprendizaje por transferencia (*transfer learning*)

El aprendizaje por transferencia es una técnica de *machine learning* que consiste en reutilizar el conocimiento y las características ya aprendidas por un modelo en una tarea inicial para mejorar el rendimiento y la generalización en una nueva tarea relacionada [7]. Este enfoque se divide principalmente en tres variantes según la relación de los datos y tareas: el *transfer* inductivo, el *transfer* no supervisado y el

transfer transductivo o adaptación de dominio [7]. A diferencia del ajuste fino (*finetuning*), que entrena un modelo existente en datos específicos para optimizar la tarea original, el aprendizaje por transferencia adapta la red a un problema nuevo [7].

Esta metodología resulta de utilidad cuando la nueva tarea cuenta con un dataset limitado, ya que mitiga el sobreajuste y reduce los tiempos de entrenamiento y los costos computacionales asociados [7, 8]. Su implementación se realiza mediante una arquitectura base donde las capas inferiores (que capturan rasgos generales) suelen mantenerse fijas o congeladas, mientras que las capas superiores se configuran como entrenables para adaptarse a los patrones del nuevo objetivo [8].

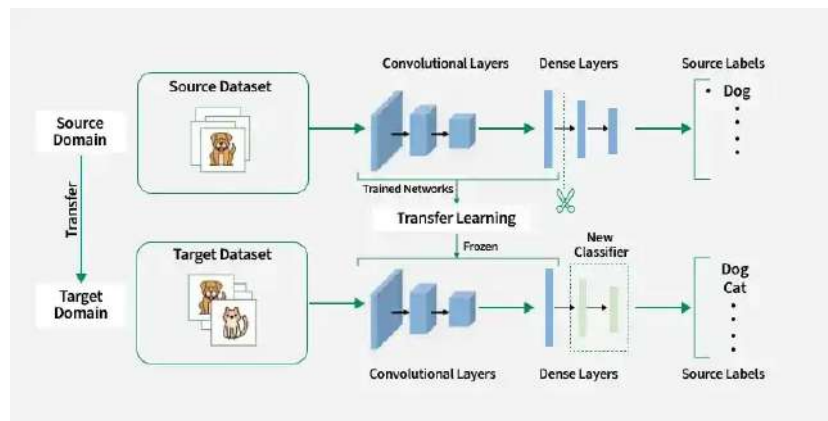


Figura 4: Estructura del Transfer Learning [8].

Detección de objetos con IA

La detección de objetos combina las subtarefas de localización (marcar el objeto mediante una caja delimitadora o *bounding box*) y clasificación (asignar una categoría semántica) para identificar qué y dónde están los elementos en una imagen o video, diferenciándose de la clasificación tradicional que etiqueta la imagen completa y de la segmentación que opera a nivel de píxel [9, 10]. Para lograrlo, los enfoques modernos de *deep learning* basados en redes neuronales convolucionales procesan la imagen a través de una arquitectura general de tres partes: un *backbone* que extrae mapas de características, un *neck* que los combina y un *head* que predice las coordenadas y las puntuaciones de clasificación. La precisión de estas predicciones se evalúa habitualmente mediante la métrica de *Intersección sobre la Unión (IoU)*, que mide el porcentaje de superposición entre la caja del modelo y la caja real (*ground truth*) [9].

Dentro del aprendizaje profundo, los algoritmos se dividen en dos grandes metodologías: los detectores de dos etapas, como la familia R-CNN (incluyendo Fast y Faster R-CNN) que primero generan propuestas de región y luego los clasifican, ofreciendo una alta precisión de localización a cambio de un mayor costo computacional; y los detectores de una sola etapa como YOLO, SSD y RetinaNet, que unifican ambos pasos para optimizar la velocidad en tiempo real [9, 10]. Por otro lado, existen técnicas de *machine learning* tradicional que ofrecen alternativas viables basadas en la selección manual de características como el algoritmo de *Viola-Jones*, las

funciones de *características de canales agregados (ACF)* y las *Máquinas de Vectores de Soporte (SVM)* que utilizan *Histogramas de Gradientes Orientados (HOG)* [10]. Ambas vertientes forman parte de sistemas que se utilizan en la actualidad en áreas como la navegación autónoma, la robótica, la videovigilancia y el diagnóstico en imágenes médicas [9, 10].

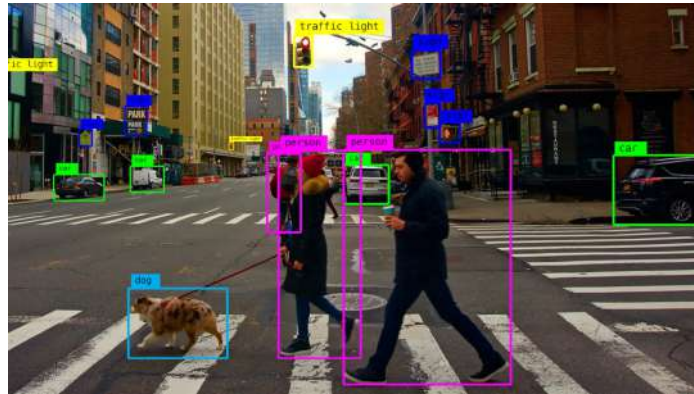


Figura 5: Ejemplo de Image Detection

<https://www.augmentedstartups.com/blog/how-to-implement-object-detection-using-deep-learning-a-step-by-step-guide?srltid=AfmBOopJJz94bIf1bvi328tY-T776Q5RM0RZcHf-MI54LcdZCP4GmNq>

YOLO

YOLO (*You Only Look Once*) es una familia de redes neuronales convolucionales (CNN) diseñada para tareas de visión artificial en tiempo real, tales como detección de objetos, segmentación de imágenes, estimación de posturas y clasificación [11, 12]. Propuesto en 2015, YOLO revolucionó el campo de la detección de objetos al abordarlo como un problema de regresión a diferencia de los detectores previos de dos etapas. YOLO requiere una sola pasada hacia adelante (*forward pass*) a través de la red, prediciendo simultáneamente cajas delimitadoras (*bounding boxes*) y las probabilidades de clase [11, 12]. Su evolución histórica abarca desde las primeras versiones con mejoras en normalización por lotes y cajas de anclaje, pasando por YOLOv8 (desarrollado por *Ultralytics*), hasta versiones más recientes como YOLOv10 y YOLO26 [11].

En cuanto a su arquitectura, YOLO procesa la imagen redimensionando y dividiéndola en una cuadrícula de celdas de tamaño $S \times S$ [12]. Si el centro de un objeto cae en una celda, esta es responsable de su detección mediante la predicción de múltiples cajas delimitadoras con sus coordenadas (x, y, w, h) y un puntaje de confianza, junto con las probabilidades condicionales de la clase [12]. El flujo interno utiliza una red troncal (*backbone*) de 24 capas convolucionales, complementada con circunvoluciones de 1×1 y 3×3 para reducir parámetros y estabilizar el cómputo, funciones de activación Leaky ReLU y capas totalmente conectadas que generan un tensor final de predicciones [12]. Durante el entrenamiento, optimiza el aprendizaje mediante una función de pérdida de error mínimo cuadrático que balancea el error de localización y el error de clasificación, aplicando técnicas de ajuste como *Batch Normalization* y *Dropout* para evitar el sobreajuste [12].

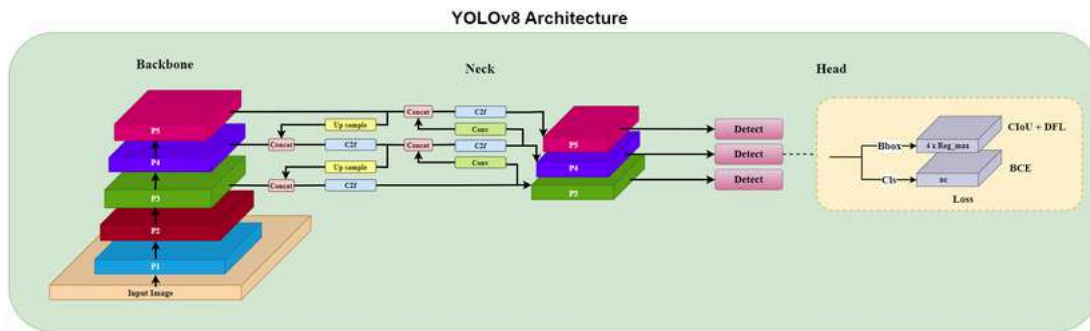


Figura 6: Arquitectura del modelo YOLOv8.

https://www.researchgate.net/figure/Basic-architecture-of-YOLOv8-object-detection-model_fig1_376831

163

mAP50

En el contexto de la evaluación de modelos de detección de objetos como *YOLO*, la métrica *mAP50* es un indicador fundamental para medir la eficacia y precisión del algoritmo [13]. *mAP50* se define y estructura de la siguiente manera:

1. *Concepto Base (mAP)*. Las siglas provienen de Mean Average Precision (Precisión Media Promedio). Esta métrica amplía el concepto de Average Precision (AP), el cual calcula el área bajo la curva de precisión-recall. El mAP calcula los valores medios de AP a través de múltiples clases de objetos, lo que proporciona una evaluación completa del rendimiento del modelo en escenarios de detección multiclase [13].
2. *Intersection over Union (IoU)*. El IoU es una medida que cuantifica la superposición entre un bounding box predicho y uno real (ground truth). Desempeña un papel fundamental en la evaluación de la precisión de la localización de objetos [13].
3. *El Umbral de 50 (IoU 0.50)*. El valor 50 indica que la métrica se calcula en un umbral estricto de Intersection over Union (IoU) de 0.50 [13].

Solución del problema

1. Generación del dataset

Utilizando la cámara web de la computadora, se tomaron fotografías de las señales con una resolución de 640×480 píxeles, mostradas en la figura 7, con una distribución aproximada de 200 apariciones por tipo de señal.

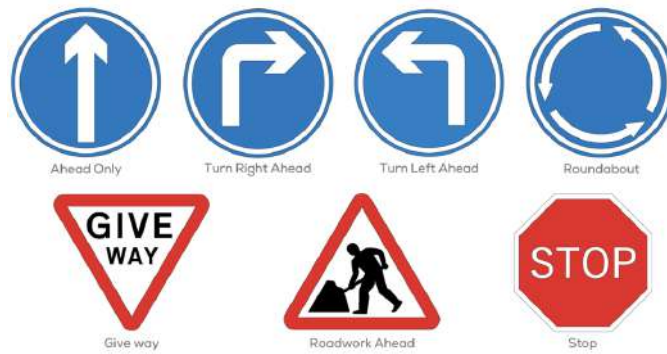


Figura 7: Señales a detectar.

Para asegurar la variedad en el dataset, durante la captura de fotografías se tomaron en cuenta los siguientes factores:

- Posición de la señal.
- Rotación de la señal.
- Tamaño de la señal.
- Iluminación ambiental.
- Aparición de múltiples señales.

2. Etiquetado de las imágenes

Utilizando *Roboflow*, se creó un proyecto de detección de objetos, y se etiquetaron las señales cargadas, utilizando las herramientas de polígono. Esto permitió que, a diferencia de las *bounding boxes* que pueden delimitar elementos de ruido junto con la señal, los polígonos únicamente delimitaban el área de las señales como se muestra en la figura 8.

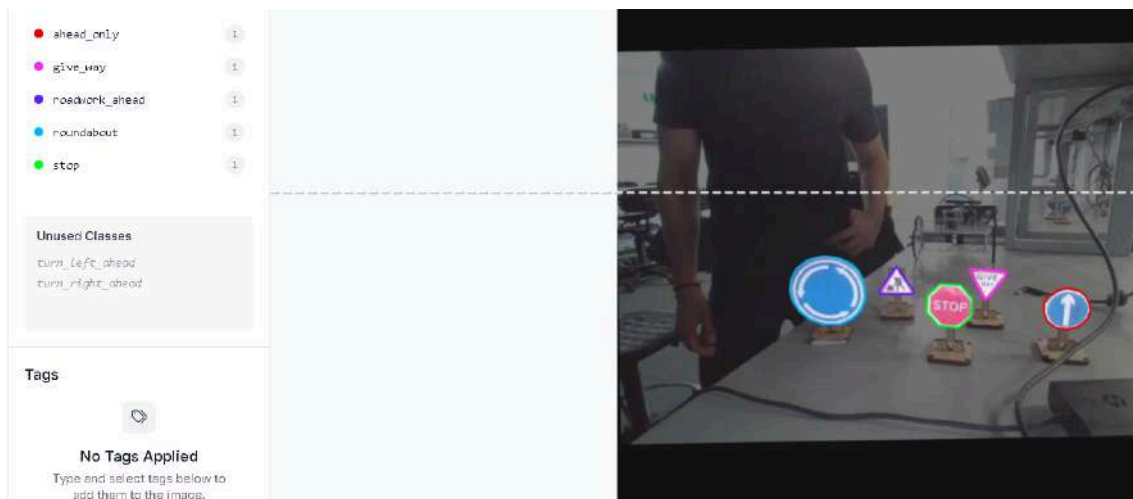


Figura 8: Etiquetado de señales con herramienta de polígono de Roboflow.

3. Aumento del dataset

Antes de realizar el data augmentation, se dividió la cantidad de imágenes que se iban a aplicar como datos de entrenamiento, de validación y de prueba. Se optó por realizar una distribución de 80% para entrenamiento, 15% para validación y 5% para prueba como se muestra en la figura 9.



Figura 9: Distribución de imágenes para entrenamiento, validación y prueba.

Posteriormente, como se muestra en la figura 10, se hizo un preprocesamiento de las imágenes, reescalándolas a una resolución de 640x640 para adaptarse a la resolución de la cámara del robot y una auto-orientación para que todas las imágenes estén orientadas de manera consistente.

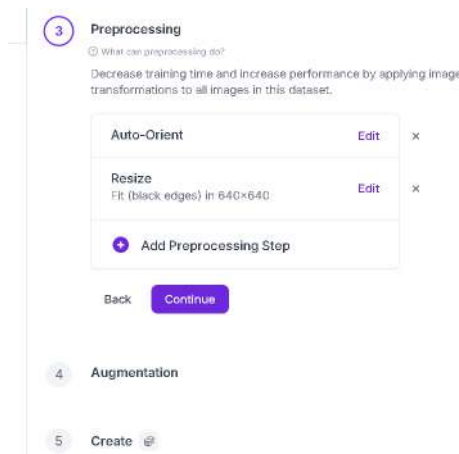


Figura 10: Preprocesamiento en Roboflow.

Utilizando las herramientas de Roboflow para data augmentation, se aplicaron las siguientes operaciones previo a la exportación del dataset:

- **Rotación:** Se aplicó una rotación entre los -5° y 5°
- **Difuminado:** Se aplicó un difuminado con una desviación estándar de 1.8px
- **Brillo:** Se ajustó el brillo de algunas imágenes del data augmentation con una exposición entre -15% y 15%

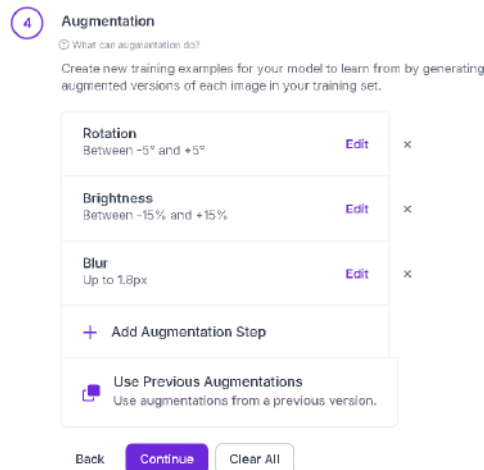


Figura 11: Operadores utilizados para data augmentation en Roboflow.

De este modo se obtuvo un dataset conformado por 2 605 imágenes etiquetadas.

4. Entrenamiento de la red con YOLO

Para el entrenamiento de la red neuronal que detectara los objetos determinados en el dataset, se utilizó la biblioteca *Ultralytics* para entrenar un modelo YOLOv8 en un entorno de la nube de *Google Colab* el cual se cambió su entorno de ejecución a que fuera con GPU T4 para mayor velocidad de procesamiento. El proceso de entrenamiento fue el siguiente:

1. Utilizando la *API key* que ofrece *Roboflow*, se importó la versión 3 del dataset que está incluido en el workspace del equipo en un formato de YOLOv8. Al importarlo de esta manera, las imágenes vienen acompañadas de un archivo `.txt` con las coordenadas de los objetos previamente seleccionados y etiquetados. También junto a un archivo `data.yaml` con las rutas de los datos de entrenamiento, validación y los nombres de las clases.
2. Después de haber instalado la biblioteca de *Ultralytics* en el entorno de la nube con `pip`, se cargó un modelo YOLOv8n (YOLOv8 Nano) que es una variante más ligera y rápida de este tipo de modelo. Al cargar un archivo `.pt` de esta manera: `model = YOLO('yolov8n.pt')`, se está aplicando Transfer Learning para usar un modelo pre entrenado con millones de imágenes de uso general. Esto para que sea más fácil detectar, formas, colores y texturas.
3. Para entrenar el modelo, se guardó en una variable `results` la función `model.train` el cual tiene los siguientes parámetros:
 - **data:** Es la variable que apunta al archivo `data.yaml` donde están guardados las rutas de las imágenes del dataset
 - **epoch:** La red neuronal se entrena con 100 épocas. Esto quiere decir que leerá el *dataset* completo 100 veces y ajustará sus pesos mediante *backpropagation*.

- **imgz:** Todas las imágenes del dataset se re escalan a un tamaño de 640×640 para que tenga consistencia en las capas de convolución y respetar la resolución a la que el *Puzzlebot* leerá las imágenes.
 - **batch:** Las imágenes se procesarán en grupos de 16. La red analizará 16 imágenes y calcula el gradiente promedio del error actualizando sus pesos.
 - **name:** Las métricas y pesos finales se guardarán en una carpeta llamada `yolo8_puzzlebot_signs_v2`.
4. Una vez pasadas las 100 épocas y entrenado el modelo, se obtienen las métricas de precisión en el subset de validación con la función `model.val()`. Este imprime el *Model Summary* para revisar la complejidad final que tiene información como el número de capas, cantidad de parámetros y operaciones de punto flotante.
 5. Por último se exportaron los resultados del entorno de la nube a la computadora local mediante un script que comprime en un archivo `.zip` la carpeta `runs` que contiene gráficas, matrices de confusión, imágenes de muestra y el modelo llamado `best.pt`.

5. Intercomunicación del Puzzlebot con la computadora

Posteriormente para verificar el funcionamiento del modelo, se realizó un nodo en ROS2 el cual se ejecuta en una computadora externa debido a que los recursos de hardware de la *Jetson Nano* del *Puzzlebot* son limitados para el uso del modelo en este.

El nodo carga el modelo del archivo `best.pt`, inicializa una matriz `self.img` con dimensiones de 480×640 y 3 canales de color BGR para recibir la imagen y se utiliza `CVBridge` para convertir el formato de imagen de ROS2 a OpenCV que es el que requiere YOLO para hacer inferencias.

La computadora externa se conecta a la misma red del *Puzzlebot* y se configura en el mismo `ROS_DOMAIN_ID` que la terminal del *Puzzlebot*. Este nodo recibe mediante el tópico `/video_source/raw` la imagen en crudo de la cámara del *Puzzlebot*.

Para realizar las inferencias, se configuró un timer que con una frecuencia de 20Hz, se ejecuta la función `self.model(self.img, conf=0.5, verbose=False)` la cual tiene el frame actual y tiene un umbral de 0.5 para ignorar detecciones que esten menores al 50% de probabilidad que sean correctas.

Para extraer las coordenadas del objeto y dibujar una Bounding Box alrededor de esta, se itera sobre los resultados y se extrae los tensores de las cajas de detección. Estos se convierten a arreglos que tienen `xmin`, `ymin` y `xmax`, `ymax`. Se usa la función `.plot()` que coloca el cuadro y etiqueta a la imagen detectada.

La imagen con el recuadro sobre el objeto detectado se publica al tópico `/yolo/inference_result` y también se envía un mensaje tipo String al tópico `/yolo/detecciones` con la etiqueta del objeto detectado.

Resultados

Tras finalizar la configuración y el entrenamiento del modelo predictivo basado en la arquitectura YOLOv8, se procedió a la evaluación del sistema. Para comprobar su viabilidad y robustez, los resultados se analizaron en tres fases principales: la consolidación del conjunto de datos, Resumen del Modelo, Métricas de Precisión, el análisis de las métricas de entrenamiento y, finalmente, las pruebas de inferencia en tiempo real.

Consolidación y Procesamiento del Dataset

En primer lugar, se garantizó un conjunto de datos altamente balanceado, mitigando el riesgo de sesgos hacia una señal en particular. Las clases originales se distribuyeron de la siguiente manera: `ahead_only` (222), `give_way` (219), `roadwork_ahead` (216), `roundabout` (216), `stop` (218), `turn_left_ahead` (238) y `turn_right_ahead` (237).

Posteriormente para fortalecer la capacidad de generalización del modelo ante las condiciones físicas de la pista, se aplicó una etapa de preprocesamiento en *Roboflow*. Las imágenes fueron reorientadas automáticamente y redimensionadas a 640x640 píxeles, utilizando relleno con bordes negros (`Fit`) para preservar la relación de aspecto de las señales. Continuando, se implementó una estrategia de `Data Augmentation` generando 3 salidas por cada ejemplo original. Las alteraciones aplicadas incluyen rotaciones espaciales aleatorias en un rango de -5° a $+5^{\circ}$. Este margen de inclinación resulta ideal, puesto que introduce un nivel de dificultad adecuado para robustecer el entrenamiento del modelo, pero sin llegar a distorsionar la perspectiva de las imágenes al punto de causar confusiones entre las clases. De igual forma, se implementaron ajustes de brillo de $\pm 15\%$ para llenar al conjunto de datos de con una amplia variedad de contrastes, así como un efecto de desenfoque de hasta 1.8 px, el cual fue diseñado para incrementar la capacidad de generalización y facilitar el aprendizaje de la red neuronal. En conjunto, estas modificaciones simulan de manera eficaz las condiciones operativas reales del sistema, tales como las variaciones lumínicas del entorno físico, el desenfoque provocado por el avance continuo del robot y la inclinación natural de la cámara, a continuación se muestran algunos ejemplos.

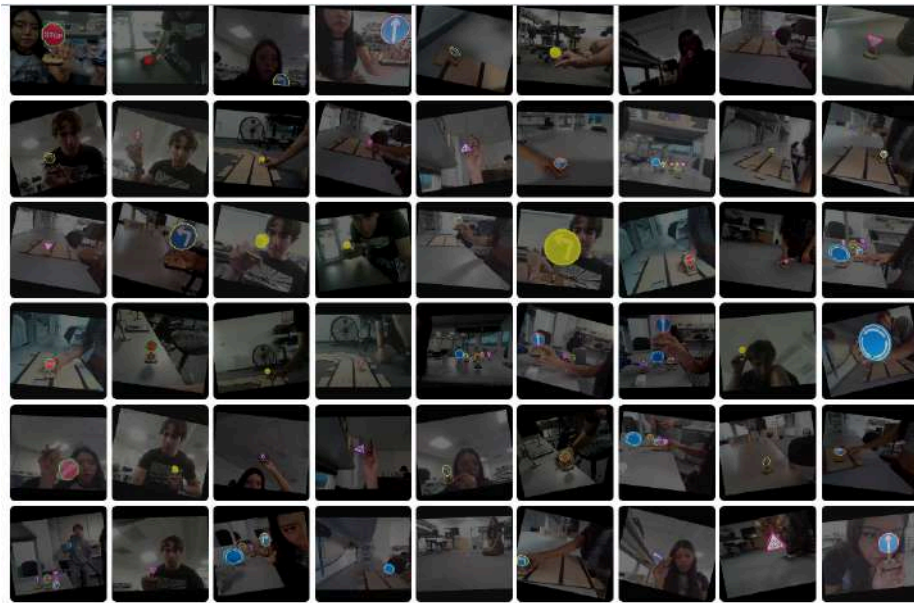


Figura 12: Ejemplo del Data Augmentation aplicado en Roboflow.

Como resultado de este aumento, el conjunto final se expandió significativamente, segmentándose en:

- **Train Set (94%)**: 2460 imágenes utilizadas para el aprendizaje puro.
- **Valid Set (4%)**: 97 imágenes para ajustar los hiperparámetros durante el entrenamiento.
- **Test Set (2%)**: 48 imágenes completamente nuevas para la evaluación final.

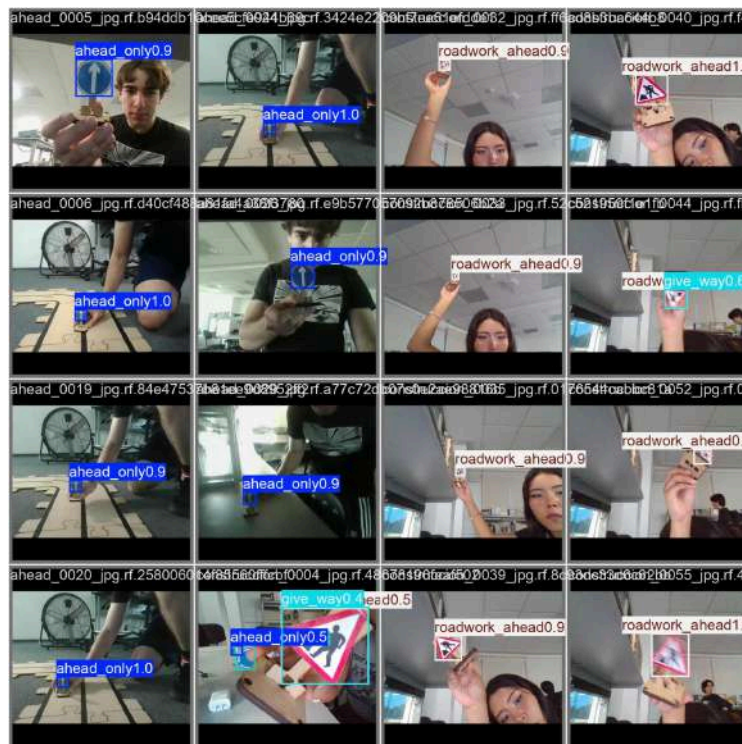


Figura 13: Muestra de los ejemplos de lotes (batches) de validación.

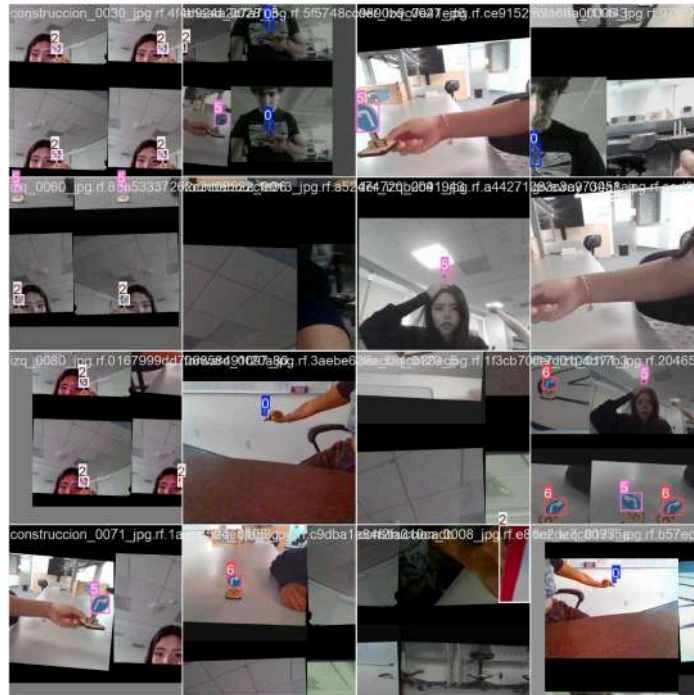


Figura 14: Muestra de los ejemplos de lotes (batches) de entrenamiento generados en Roboflow, evidenciando las técnicas de aumento de datos aplicadas.

Resumen del Modelo

En lo que respecta al resumen general del modelo, los indicadores cuantitativos confirman que la arquitectura implementada logró un entendimiento profundo de las características visuales de cada señal de tránsito. El proceso de convergencia obtenido demuestra que la red neuronal no solo memorizó los patrones del conjunto de entrenamiento, sino que adquirió la capacidad de generalizar su conocimiento ante escenarios de validación completamente nuevos. Esto se ve respaldado de manera directa por el valor de cada una de las clases las cuales se posicionan en un sobresaliente 91% - 99. % bajo un umbral de intersección sobre unión de 0.50. Dicho porcentaje es un claro indicativo de que el acoplamiento entre la etapa de aumento de datos en *Roboflow* y la estructura convolucional de la red neuronal fue altamente efectivo, dando como resultado un sistema robusto y optimizado para las demandas computacionales en tiempo real del Puzzlebot.

```

Ultralytics 8.4.53 Python-3.12.13 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
Model summary (fused): 73 layers, 3,007,013 parameters, 0 gradients, 8.1 GFLOPs
val: Fast image access (ping: 0.0±0.0 ms, read: 1144.9±342.7 MB/s, size: 32.0 KB)
val: Scanning /content/imissnwjns-3/valid/labels.cache... 97 images, 0 backgrounds, 0 corrupt: 100% 97/97 37.0Mit/s 0.0s

```

Class	Images	Instances	Box(P)	R	mAP50	mAP50-95)
all	97	135	0.938	0.955	0.967	0.883
ahead_only	21	21	0.954	0.997	0.993	0.925
give_way	24	24	0.915	0.958	0.955	0.861
roadwork_ahead	27	27	0.963	0.963	0.958	0.734
roundabout	15	15	1	0.898	0.991	0.954
stop	22	22	0.98	1	0.995	0.927
turn_left_ahead	10	10	0.823	0.935	0.918	0.849
turn_right_ahead	16	16	0.928	0.938	0.959	0.93

```

Speed: 5.6ms preprocess, 6.2ms inference, 0.0ms loss, 2.1ms postprocess per image
Results saved to /content/runs/detect/val

```

Figura 15: Resumen del modelo para cada clase.

Por el otro lado en la figura 16 en lo que respecta a la estructura interna de la red neuronal, el análisis topológico revela una arquitectura profunda y altamente optimizada conformada por un total de 3,012,213 parámetros aprendibles. El diseño del sistema se basa en una red neuronal convolucional que emplea múltiples capas de convolución bidimensional para la extracción iterativa de características visuales complejas. Para garantizar la estabilidad matemática y acelerar la convergencia durante el entrenamiento, estas extracciones son procesadas mediante capas de normalización por lotes a lo largo de toda la red. Esta configuración resulta idónea para su implementación en el hardware del Puzzlebot, ya que mantiene una huella computacional lo suficientemente ligera como para ejecutar inferencias a veinte cuadros por segundo sin sacrificar la robustez predictiva necesaria para la navegación autónoma.

Resumen del Modelo: best2.pt					
Layer ID	Layer Name	Type / Class	Parameters		
3	model.0.conv	Conv2d	432		
4	model.0.bn	BatchNorm2d	32		
7	model.1.conv	Conv2d	4,608		
8	model.1.bn	BatchNorm2d	64		
11	model.2.cv1.conv	Conv2d	1,824		
12	model.2.cv1.bn	BatchNorm2d	64		
14	model.2.cv2.conv	Conv2d	1,536		
15	model.2.cv2.bn	BatchNorm2d	64		
19	model.2.m.0.cv1.conv	Conv2d	2,304		
20	model.2.m.0.cv1.bn	BatchNorm2d	32		
22	model.2.m.0.cv2.conv	Conv2d	2,304		
23	model.2.m.0.cv2.bn	BatchNorm2d	32		
25	model.3.conv	Conv2d	18,432		
26	model.3.bn	BatchNorm2d	128		
29	model.4.cv1.conv	Conv2d	4,096		
30	model.4.cv1.bn	BatchNorm2d	128		
32	model.4.cv2.conv	Conv2d	8,192		
33	model.4.cv2.bn	BatchNorm2d	128		
37	model.4.m.0.cv1.conv	Conv2d	9,216		
38	model.4.m.0.cv1.bn	BatchNorm2d	64		
40	model.4.m.0.cv2.conv	Conv2d	9,216		
41	model.4.m.0.cv2.bn	BatchNorm2d	64		
44	model.4.m.1.cv1.conv	Conv2d	9,216		
45	model.4.m.1.cv1.bn	BatchNorm2d	64		
47	model.4.m.1.cv2.conv	Conv2d	9,216		
48	model.4.m.1.cv2.bn	BatchNorm2d	64		
50	model.5.conv	Conv2d	73,728		
51	model.5.bn	BatchNorm2d	256		
54	model.6.cv1.conv	Conv2d	16,384		
55	model.6.cv1.bn	BatchNorm2d	256		
57	model.6.cv2.conv	Conv2d	32,768		
58	model.6.cv2.bn	BatchNorm2d	256		

Figura 16: Resumen del modelo general.

Métricas de Precisión (Precision Metrics)

La evaluación de las métricas de precisión confirma la alta confiabilidad del sistema en la clasificación de señales viales al minimizar la incidencia de falsos positivos. El modelo alcanzó un desempeño casi perfecto con una precisión general de 0.9378, lo cual es vital para garantizar que el robot no ejecute maniobras erróneas ni paros de emergencia innecesarios.


```

=== MÉTRICAS DE PRECISIÓN ===
mAP50: 0.9672
mAP50-95: 0.8830
Precisión (Precision): 0.9378
Exhaustividad (Recall): 0.9555

```

Figura 17: Métricas de precisión extraídas del entrenamiento.

Rendimiento del Modelo y Métricas de Entrenamiento

Durante la fase de entrenamiento, el modelo demostró una convergencia estable y exitosa. El análisis de las gráficas de pérdida arroja resultados muy favorables. Por un lado, la gráfica de pérdida de caja (`box_loss`, tanto en validación como en entrenamiento) muestra una tendencia descendente continua, lo que indica que el algoritmo aprendió a delimitar de forma precisa las señales con sus Bounding Boxes. Por otro lado, la pérdida de clasificación (`cls_loss`) disminuyó de forma análoga, confirmando que el sistema identifica correctamente a qué clase pertenece cada objeto detectado, sin presentar indicios graves de sobreajuste (*overfitting*).

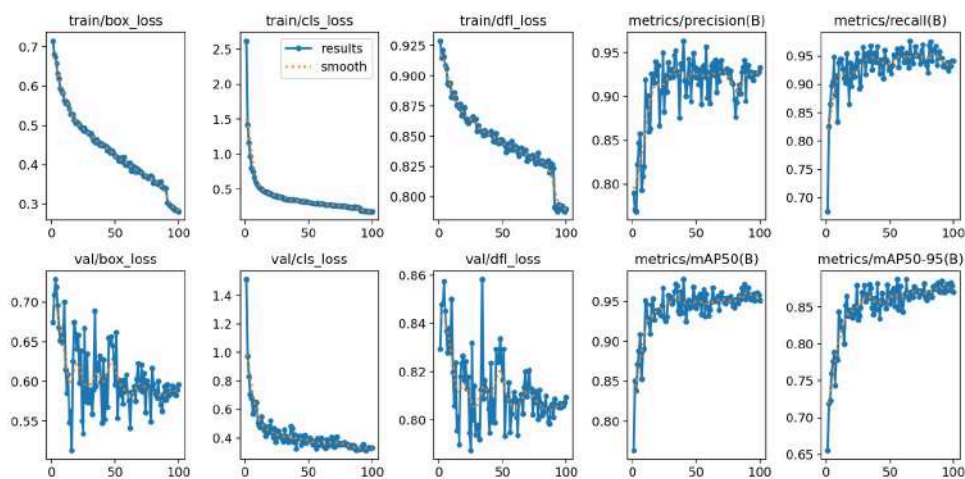


Figura 18: Curvas de aprendizaje y métricas de rendimiento extraídas del entrenamiento. Se observa la estabilización de las pérdidas y el incremento del `mAP50`.

Asimismo, como se observa en la figura 18 las métricas clave de evaluación general alcanzaron valores óptimos. La precisión (`Precision`) indica una tasa muy baja de falsos positivos, mientras que la sensibilidad (`Recall`) confirma que el modelo rara vez omite una señal presente en el fotograma. Esto se consolida en la métrica `mAP50` (Mean Average Precision a un umbral de Intersección sobre Unión del 50%), la cual se mantuvo alta y estable durante las últimas épocas, garantizando confiabilidad en el despliegue.

Adicionalmente, la matriz de confusión corrobora el alto nivel de certidumbre del modelo. La concentración de valores altos en la diagonal principal demuestra que la red neuronal distingue correctamente entre clases geométricamente similares (como

turn_left_ahead y turn_right_ahead), minimizando las clasificaciones erróneas.

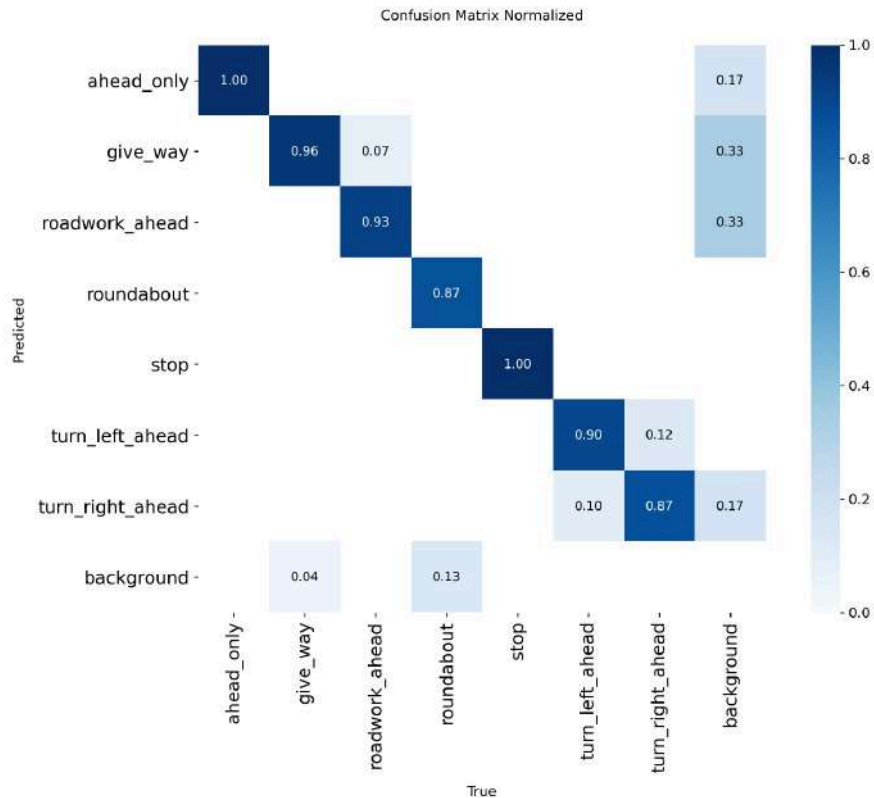
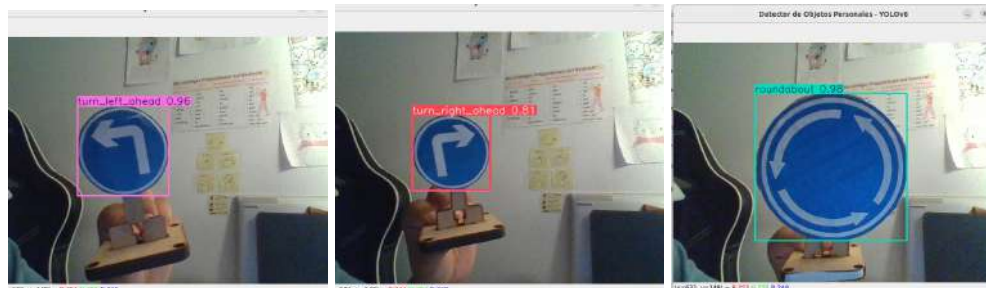


Figura 19: Matriz de confusión del modelo validado, mostrando la alta tasa de aciertos y la baja confusión entre las distintas señales de tránsito.

Pruebas de Inferencia Práctica y Despliegue en ROS 2

Finalmente, tras validar el rendimiento teórico, el modelo (`best.pt`) fue sometido a pruebas de inferencia práctica. Para llevar a cabo una transición segura, la primera fase de despliegue se realizó utilizando la cámara web local de la computadora base. Esta prueba permitió verificar que el procesamiento en tiempo real se ejecutaba sin caídas de rendimiento (latencia) y que los tensores de la red procesaban correctamente los fotogramas del entorno local.



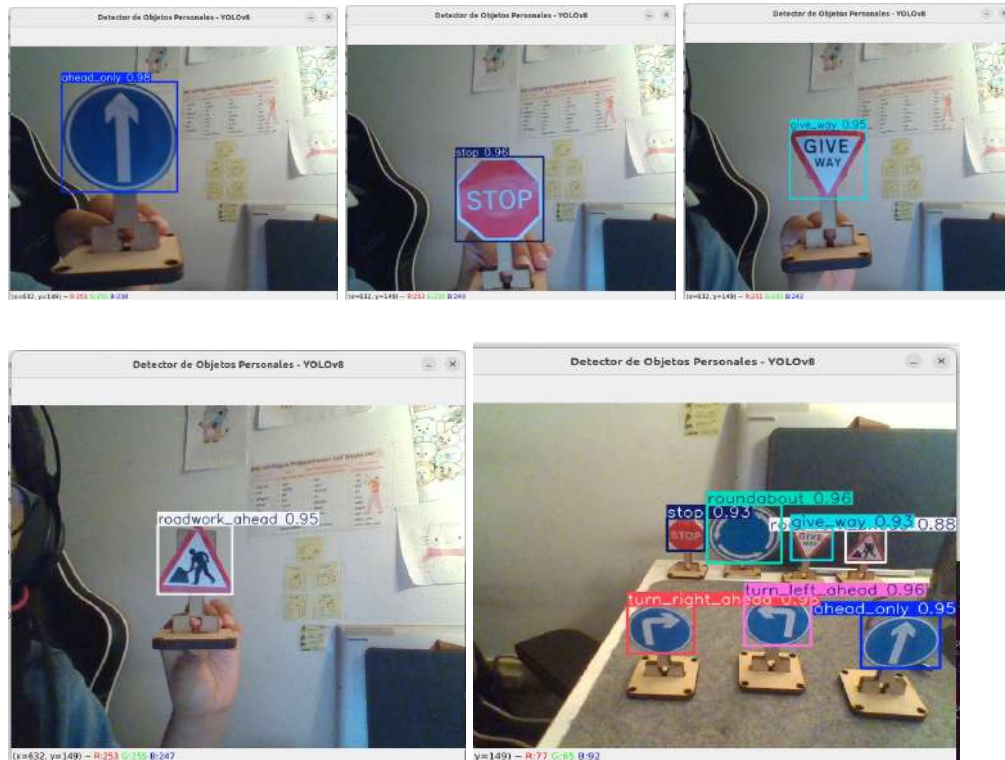


Figura 20: Validación inicial de inferencia en tiempo real utilizando la cámara local de la estación de control.

Una vez comprobada la eficacia del algoritmo a nivel de software, se procedió con la integración física sobre el Puzzlebot. Mediante el framework ROS 2, la computadora actuó como un nodo remoto, procesando el tópico de video proveniente de la cámara del robot. Los resultados fueron sumamente satisfactorios: el sistema detectó las señales con altos niveles de confianza (por encima del 0.8 o 80%), incluso mientras el robot se encontraba en movimiento, publicando los resultados correctamente en los tópicos correspondientes para la toma de decisiones.



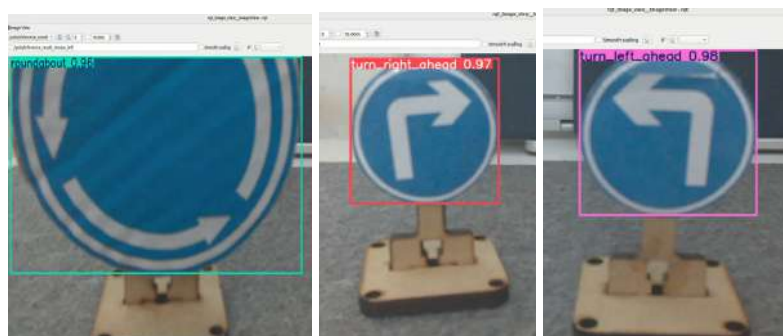


Figura 21: Despliegue exitoso operando con la telemetría visual del Puzzlebot, detectando y delimitando las señales en tiempo real.

Finalmente, se comparte un video demostrativo de la detección de señales por el Puzzlebot:

https://drive.google.com/file/d/16wjiRZKkOGmgwX4ICO3OMHY1d_ck5fBg/view?usp=sharing

Conclusiones

Finalmente, se comprobó con éxito el funcionamiento del modelo de visión cumpliendo los objetivos con la creación de una red neuronal convolucional, la recolección manual y etiquetado de fotos para las siete señales de tránsito, el ajuste de los parámetros de data augmentation y de entrenamiento para el correcto aprendizaje del modelo. El sistema logró cumplir satisfactoriamente los objetivos planteados los resultados cuantitativos, respaldados por una precisión global de 0.9378 y un alto valor de precisión media promedio, demuestran que la red neuronal convolucional desarrolló una sólida capacidad de generalización frente a variaciones lumínicas, desenfoque y cambios de perspectiva.

El sistema destaca por su sobresaliente certidumbre para clasificar directivas críticas con una efectividad casi perfecta, lo cual minimiza los falsos positivos y garantiza una navegación autónoma segura. Sin embargo, presenta un margen de mejora en la discriminación de señales direccionales bajo acercamientos extremos durante el movimiento rápido del robot. Para superar estas limitaciones y optimizar el rendimiento futuro, se propone enriquecer el conjunto de entrenamiento con muestras y aumentos de datos más robustos enfocados en dichas clases, así como realizar un ajuste fino de los hiperparámetros. Finalmente, la integración de un algoritmo de seguimiento espacial de objetos permitiría estabilizar las detecciones continuas, mitigando las pérdidas temporales por el parpadeo de la cámara y consolidando un sistema de percepción visual verdaderamente tolerante a fallos.

Próximamente, se podrá implementar una estrategia de control en el que el robot tome decisiones con base a la señal detectada durante su trayecto, construyendo así un sistema de navegación autónoma.

Referencias

- [1] Bergmann, D. (2025, 26 noviembre). *¿Qué es el deep learning?* IBM Think.
<https://www.ibm.com/mx-es/think/topics/deep-learning>
- [2] Wang, J. (2026, 15 mayo). *Deep Learning*. MathWorks.
<https://la.mathworks.com/discovery/deep-learning.html>
- [3] IBM. (2025, 27 noviembre). *¿Qué son las redes neuronales convolucionales?* IBM Think. <https://www.ibm.com/es-es/think/topics/convolutional-neural-networks>
- [4] GeeksforGeeks. (2026b, mayo 12). *Introduction to Convolution Neural Network*. GeeksforGeeks.
<https://www.geeksforgeeks.org/machine-learning/introduction-convolution-neural-network/>
- [5] Murel, J., & Kavlakoglu, E. (2025c, noviembre 26). *¿Qué es el aumento de datos?* IBM Think. <https://www.ibm.com/mx-es/think/topics/data-augmentation>
- [6] GeeksforGeeks. (2025a, julio 23). *What is Data Augmentation? How Does Data Augmentation Work for Images?* GeeksforGeeks.
<https://www.geeksforgeeks.org/computer-vision/what-is-data-augmentation-how-does-data-augmentation-work-for-images/>
- [7] Murel, J., & Kavlakoglu, E. (2025b, noviembre 17). *What is transfer learning?* IBM Think. <https://www.ibm.com/think/topics/transfer-learning>
- [8] GeeksforGeeks. (2026a, mayo 9). *Introduction to transfer learning*. GeeksforGeeks.
<https://www.geeksforgeeks.org/machine-learning/ml-introduction-to-transfer-learning/>
- [9] Murel, J., & Kavlakoglu, E. (2025a, noviembre 17). *What is object detection?* IBM Think. <https://www.ibm.com/think/topics/object-detection>

[10] Vemulapalli, K. (2026, 18 mayo). *What Is Object Detection?* MathWorks.

<https://la.mathworks.com/discovery/object-detection.html>

[11] Ultralytics. (2023, 12 noviembre). *Home | Ultralytics Docs*. Ultralytics Docs.

<https://docs.ultralytics.com/>

[12] GeeksforGeeks. (2025b, noviembre 14). *YOLO : You only look once Real time object detection*. GeeksforGeeks.

<https://www.geeksforgeeks.org/machine-learning/yolo-you-only-look-once-real-time-object-detection/>

[13] Ultralytics. (2023, 12 de noviembre). *Métricas de rendimiento de YOLO*.

Ultralytics Docs.

<https://docs.ultralytics.com/es/guides/yolo-performance-metrics>